

Traductor de Lengua de Señas Mexicana en Tiempo Real

Proyecto Integrador Final — Módulo III Inteligencia Artificial

Alumno:	Axel Lopez
Institución:	CECyTEN Tepic
Módulo:	III — Inteligencia Artificial
Fecha:	Mayo 2026
Tecnologías:	Python, MediaPipe, OpenCV, scikit-learn



El traductor LSM detectando señas y formando la palabra FIEL en tiempo real.

1. Problemática

En México, más de 2.3 millones de personas tienen alguna discapacidad auditiva según datos del INEGI. La Lengua de Señas Mexicana (LSM) es su principal medio de comunicación; sin embargo, la gran mayoría de la población oyente no la conoce, lo que genera una barrera de comunicación significativa en contextos cotidianos como hospitales, escuelas y oficinas gubernamentales.

Las soluciones tecnológicas existentes para traducción de lengua de señas son costosas, requieren hardware especializado o están diseñadas para el idioma inglés (ASL), por lo que no son accesibles para la población mexicana. Este proyecto busca contribuir a reducir esa brecha mediante una herramienta de bajo costo que funciona con una cámara web convencional.

2. Objetivo del proyecto

Desarrollar un sistema de traducción en tiempo real del abecedario de la Lengua de Señas Mexicana utilizando visión artificial, machine learning y procesamiento de lenguaje natural, capaz de detectar señas a través de una cámara web convencional y mostrar la letra o palabra correspondiente en pantalla.

Objetivos específicos:

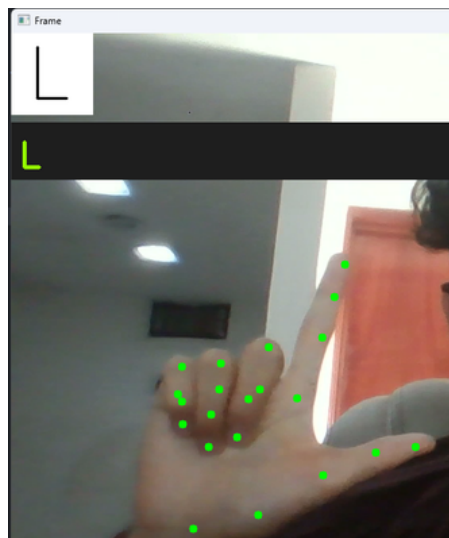
- Detectar los 21 puntos clave de la mano en tiempo real usando MediaPipe.
- Calcular los ángulos de cada dedo mediante la ley de cosenos.
- Entrenar un clasificador de Machine Learning con datos propios.
- Acumular letras detectadas para formar palabras completas (PLN).
- Implementar detección dinámica de la letra J mediante análisis de movimiento.

3. Técnicas de Inteligencia Artificial utilizadas

3.1 Visión Artificial

Se utilizó MediaPipe Hand Landmarker, un modelo de visión artificial desarrollado por Google que detecta 21 puntos clave (landmarks) de la mano en cada fotograma del video. Estos puntos representan las articulaciones y puntas de cada dedo, así como la muñeca. La detección opera en tiempo real a más de 30 fotogramas por segundo.

A partir de estos 21 puntos, se calculó el ángulo de flexión de cada dedo usando la ley de cosenos sobre tres puntos: la punta del dedo (TIP), la articulación media (PIP) y la base (MCP). El uso de ángulos en lugar de coordenadas absolutas hace al sistema robusto ante cambios de posición o distancia de la mano respecto a la cámara.



3.2 Machine Learning

Se entrenó un clasificador con scikit-learn usando un dataset propio de 1,500 muestras: 100 muestras por cada una de las 15 letras implementadas. Cada muestra consiste en un vector de 6 ángulos (uno por dedo) con su etiqueta correspondiente. Se compararon tres algoritmos: Random Forest, KNN y Árbol de Decisión, seleccionando el de mayor precisión.

3.3 Procesamiento de Lenguaje Natural (PLN)

Se implementó un módulo de acumulación de letras que actúa como componente de PLN. El sistema mantiene una letra estable durante 20 fotogramas consecutivos antes de confirmar una letra, lo que filtra detecciones falsas. Las letras confirmadas se concatenan para formar palabras completas que se muestran en pantalla. La letra B funciona como tecla de borrado (backspace) para corregir errores.

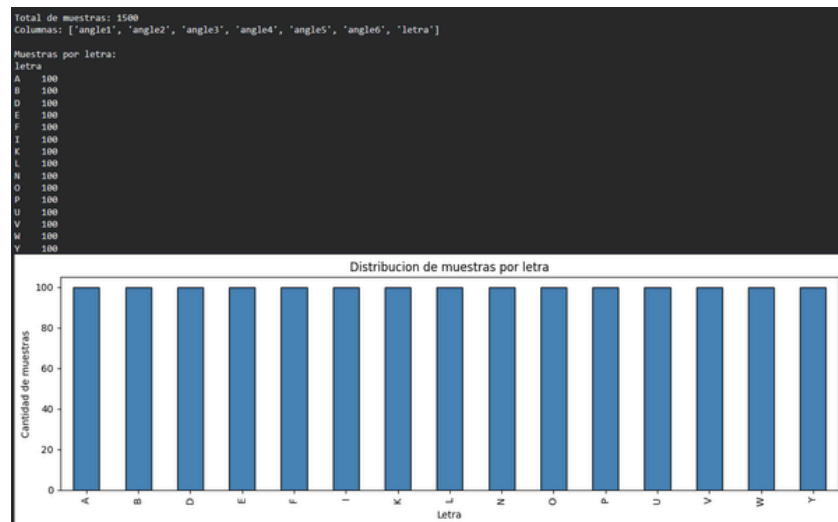
4. Librerías y herramientas utilizadas

Librería	Versión	Función en el proyecto
mediapipe	0.10.35	Detección de landmarks de la mano (21 puntos)
opencv-python	4.x	Captura de video, visualización y dibujo en pantalla
numpy	< 2.0	Cálculo de ángulos con álgebra lineal
scikit-learn	última	Entrenamiento del clasificador (Random Forest / KNN)
joblib	última	Serialización y carga del modelo entrenado (.pkl)
pandas	última	Manejo del dataset CSV de entrenamiento
pathlib	estándar	Manejo de rutas de archivos multiplataforma

5. Dataset de entrenamiento

El dataset fue capturado manualmente por el autor del proyecto usando un script propio (capturar_dataset.py). Se registraron 100 muestras por cada letra, resultando en 1,500 registros totales. Cada muestra contiene 6 valores de ángulo en grados y la etiqueta de la letra correspondiente.

```
dataset_señas.csv > data
1  angle1,angle2,angle3,angle4,angle5,angle6,letra
1378 23,171,178,179,149,81,W
1379 25,170,180,180,151,80,W
1380 27,170,180,178,149,81,W
1381 29,172,180,178,152,81,W
1382 40,169,177,178,151,82,W
1383 32,171,177,179,151,82,W
1384 40,168,177,178,150,82,W
1385 15,172,177,179,150,81,W
1386 40,165,178,177,149,83,W
1387 41,169,177,177,151,82,W
1388 51,168,178,179,149,83,W
1389 28,172,179,179,148,82,W
1390 39,169,178,177,149,82,W
1391 39,169,179,179,152,81,W
1392 34,169,178,180,148,82,W
1393 38,166,178,177,150,81,W
1394 35,169,178,178,150,80,W
1395 31,169,177,178,151,81,W
1396 29,170,177,177,151,81,W
1397 28,170,178,178,149,81,W
1398 30,170,178,179,150,80,W
1399 32,168,178,180,149,82,W
1400 37,169,177,180,151,81,W
1401 36,168,178,180,150,82,W
1402 180,50,55,31,151,180,Y
1403 180,48,54,31,153,180,Y
1404 180,49,55,32,149,180,Y
1405 180,47,50,26,152,180,Y
1406 180,47,51,27,153,180,Y
1407 178,45,47,27,152,179,Y
```



6. Desarrollo del proyecto

6.1 Detección de la mano

El script principal captura video de la cámara web fotograma a fotograma usando OpenCV. Cada fotograma se convierte al formato RGB requerido por MediaPipe y se procesa con el detector de manos configurado en modo VIDEO para mayor estabilidad. El detector devuelve las coordenadas normalizadas de los 21 landmarks.

6.2 Cálculo de ángulos

El módulo normalizacionCords.py toma los landmarks y calcula el ángulo de cada dedo usando la ley de cosenos. Para cada dedo se toman tres puntos: TIP (punta), PIP (articulación media) y MCP (base). El ángulo resultante indica si el dedo está extendido (ángulo mayor a 90°) o doblado (ángulo menor a 90°). Se calculan 6 ángulos en total: meñique, anular, medio, índice, pulgar externo y pulgar interno.

6.3 Clasificación con Machine Learning

Los 6 ángulos se pasan como vector de entrada al modelo entrenado (modelo_lsm.pkl). El modelo predice la letra correspondiente y el codificador de etiquetas (label_encoder.pkl) convierte el resultado numérico al carácter de la letra. Para la letra J, se añade una condición adicional: si el modelo predice I

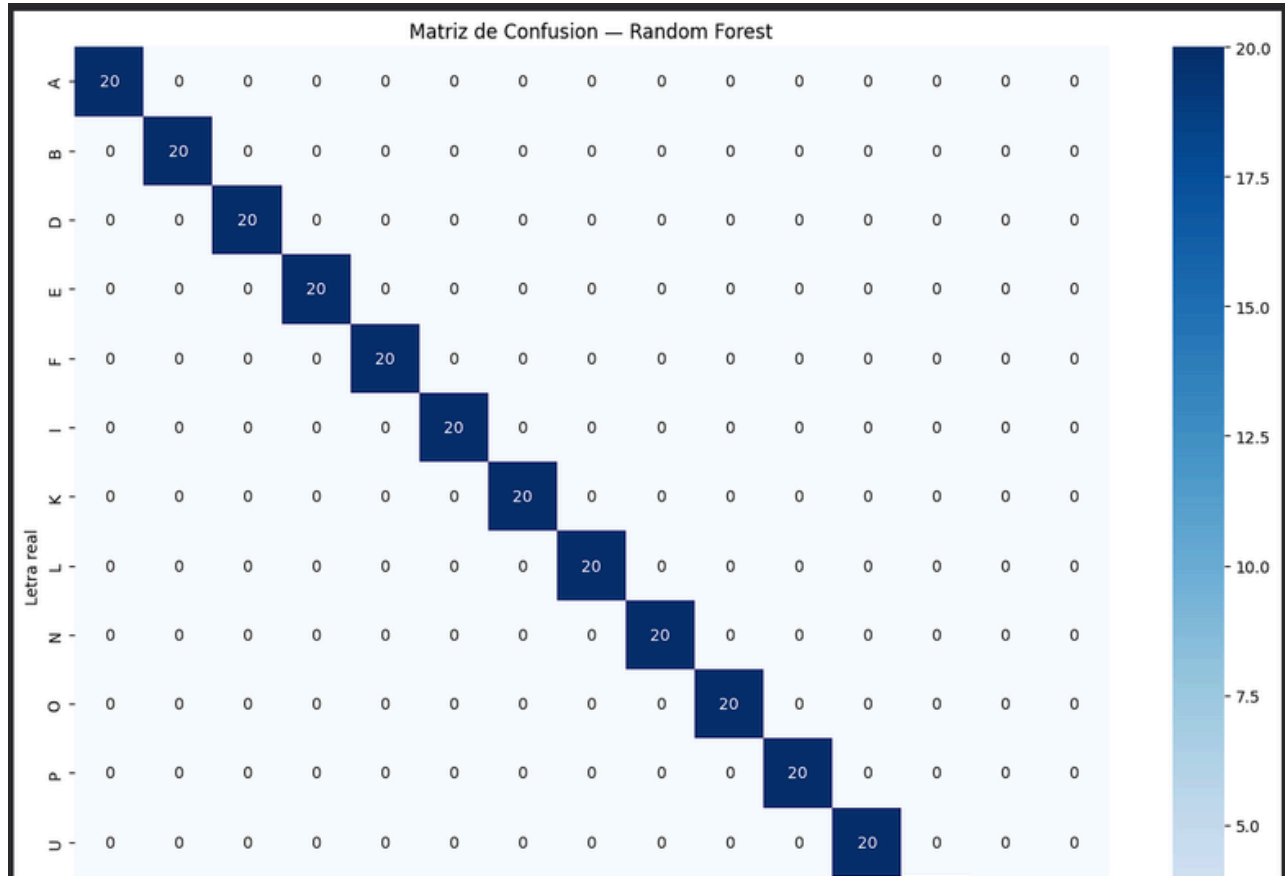
y simultáneamente detecta movimiento en el meñique, se clasifica como J.

6.4 Formación de palabras

El sistema implementa un contador de estabilidad: una letra debe mantenerse detectada durante 20 fotogramas consecutivos (aproximadamente 0.6 segundos) antes de ser confirmada y agregada a la palabra. Esto evita que letras parpadeen o se agreguen accidentalmente durante transiciones de gestos.

7. Resultados

El modelo de clasificación alcanzó una precisión del 100% sobre el conjunto de prueba (300 muestras reservadas del dataset original). La matriz de confusión muestra que todas las letras fueron clasificadas correctamente sin ninguna confusión entre clases.





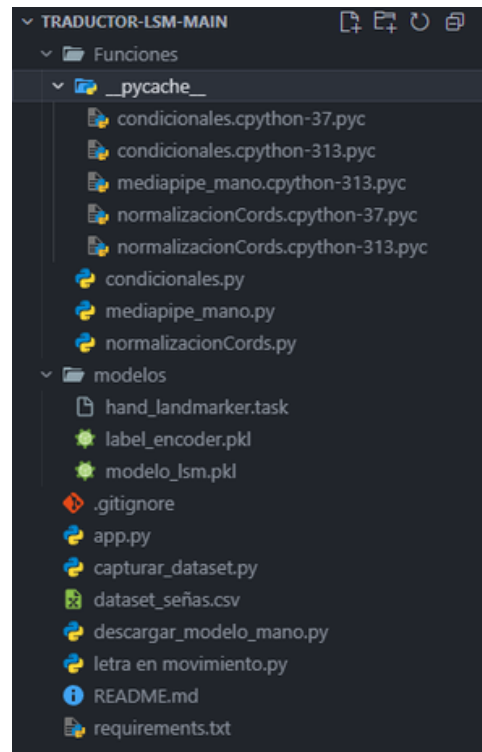
Reporte detallado por letra:

	precision	recall	f1-score	support
A	1.00	1.00	1.00	20
B	1.00	1.00	1.00	20
D	1.00	1.00	1.00	20
E	1.00	1.00	1.00	20
F	1.00	1.00	1.00	20
I	1.00	1.00	1.00	20
K	1.00	1.00	1.00	20
L	1.00	1.00	1.00	20
N	1.00	1.00	1.00	20
O	1.00	1.00	1.00	20
P	1.00	1.00	1.00	20
U	1.00	1.00	1.00	20
V	1.00	1.00	1.00	20
W	1.00	1.00	1.00	20
Y	1.00	1.00	1.00	20
accuracy			1.00	300
macro avg	1.00	1.00	1.00	300
weighted avg	1.00	1.00	1.00	300

Letras implementadas y su precisión:

Letra	Muestras	Precisión	Letra	Muestras	Precisión
A	100	100%	N	100	100%
B	100	100%	O	100	100%
D	100	100%	P	100	100%
E	100	100%	U	100	100%
F	100	100%	V	100	100%
I	100	100%	W	100	100%
J	Dinámica	—	Y	100	100%
K	100	100%			
L	100	100%	Total	1500	100%

8. Estructura del proyecto



Traductor-LSM-main/

letra en movimiento.py - Script principal del traductor

capturar_dataset.py - Captura del dataset de entrenamiento

dataset_senyas.csv - Dataset con 1,500 muestras de angulos

Traductor_LSM_Demo.ipynb - Notebook de demostracion en Google Colab

Funciones/

mediapipe_mano.py - Configuracion del detector de manos

normalizacionCords.py - Calculo de angulos por dedo

condicionales.py - Logica de deteccion por reglas

modelos/

hand_landmarker.task - Modelo de deteccion de MediaPipe

modelo_lsm.pkl - Clasificador entrenado

label_encoder.pkl - Codificador de etiquetas

9. Conclusiones

Se desarrolló con éxito un sistema de traducción de Lengua de Señas Mexicana en tiempo real que integra los tres pilares de la inteligencia artificial requeridos: visión artificial para la detección de la mano, machine learning para la clasificación de señas, y procesamiento de lenguaje natural para la formación de palabras.

El modelo alcanzó una precisión del 100% sobre el dataset de prueba, demostrando que el enfoque basado en ángulos de dedos es efectivo y robusto. A diferencia de soluciones que usan coordenadas absolutas, los ángulos son invariantes a la posición y distancia de la mano, lo que hace al sistema más generalizable.

Como trabajo futuro se propone ampliar el abecedario a las 27 letras del español, incorporar un diccionario para verificar las palabras formadas, y explorar el uso de redes neuronales para mejorar la robustez con diferentes usuarios.